

ITEC 323 — Web and Mobile Application Development

Assessment 3: Dynamic Website Development

HealthConnect Patient Portal

Student Name: Nishant Shrestha

Student ID: S00392722

Unit: ITEC 323 — Web and Mobile Application Development

Assessment: Assessment 3 (40%)

Date: 07 June 2026

Business: HealthConnect — Healthcare Patient Portal

Tech Stack: ASP.NET Core 10 MVC, SQL Server 2025, Tailwind CSS

1. Introduction

For this assessment, I built a website called HealthConnect — a patient portal for a healthcare clinic. The idea is that patients can create an account, find doctors, book appointments, and keep track of their medical visits all in one place. Admins like clinic staff can log in separately and manage appointments and patient accounts.

I chose the healthcare theme because it gave me the chance to build something realistic and useful. The Australian Digital Health Agency (2023) highlights that digital health tools are increasingly being adopted to reduce administrative burden in clinical settings, which is exactly the problem HealthConnect addresses. It also let me include features like role-based login, appointment booking with validation, and a proper admin panel which covers all the assessment requirements.

1.1 What the Website Does

- Patients can register, log in, and manage their profile
- Patients can search for doctors by specialty and book appointments
- Patients can view, edit, and cancel their appointments
- Admins can confirm or cancel appointments and manage patients
- The site works on mobile and desktop and supports dark mode
- All pages have proper security with no SQL injection and no CSRF attacks

1.2 Technologies Used

ASP.NET Core 10 MVC (C#) — This is what the unit is based on. Microsoft (2024a) describes ASP.NET Core MVC as a lightweight, cross-platform framework that separates concerns across Controllers, Models, and Views.

SQL Server 2025 Express — Free database that works well with .NET. I installed it locally with instance name NISHANT\SQLEXPRESS (Microsoft, 2025).

Tailwind CSS — Makes it easy to style without writing lots of CSS. Handles dark mode and responsive layouts well (Tailwind Labs, 2024).

BCrypt.Net — Used to hash passwords so they are stored securely in the database (BCrypt.Net, 2023).

ADO.NET — Used to run SQL queries safely using parameters which prevents SQL injection (OWASP Foundation, 2021).

VS Code and SSMS — VS Code for coding and SQL Server Management Studio for managing the database.

2. Website Screenshots

The following screenshots show the main pages of the HealthConnect website running on localhost.

2.1 Home Page

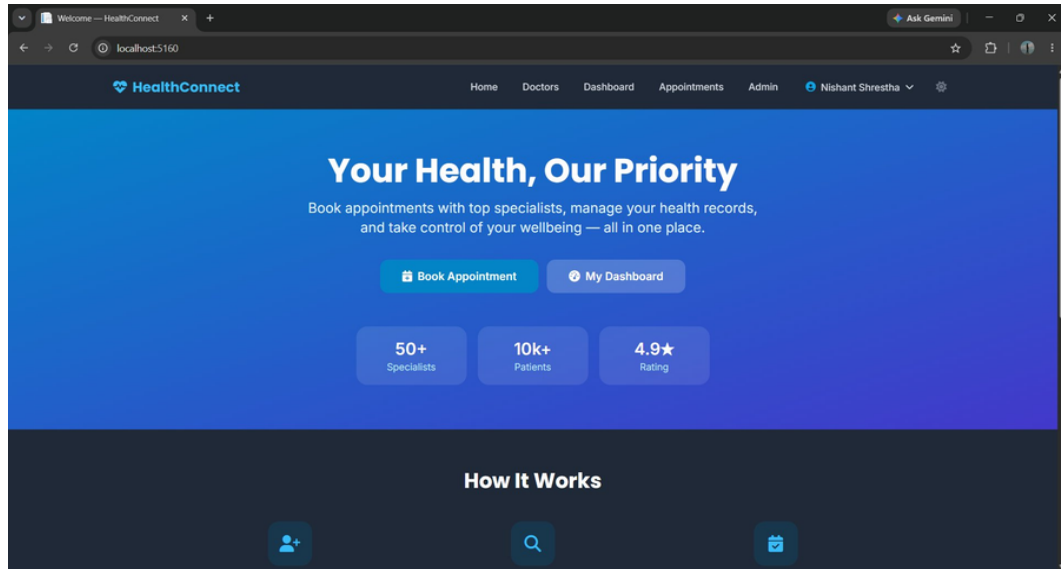


Figure 1. HealthConnect home page with hero section and navigation bar.

The home page has a gradient hero section with two call-to-action buttons and stats showing 50+ specialists, 10k+ patients, and a 4.9 star rating.

2.2 Register Page

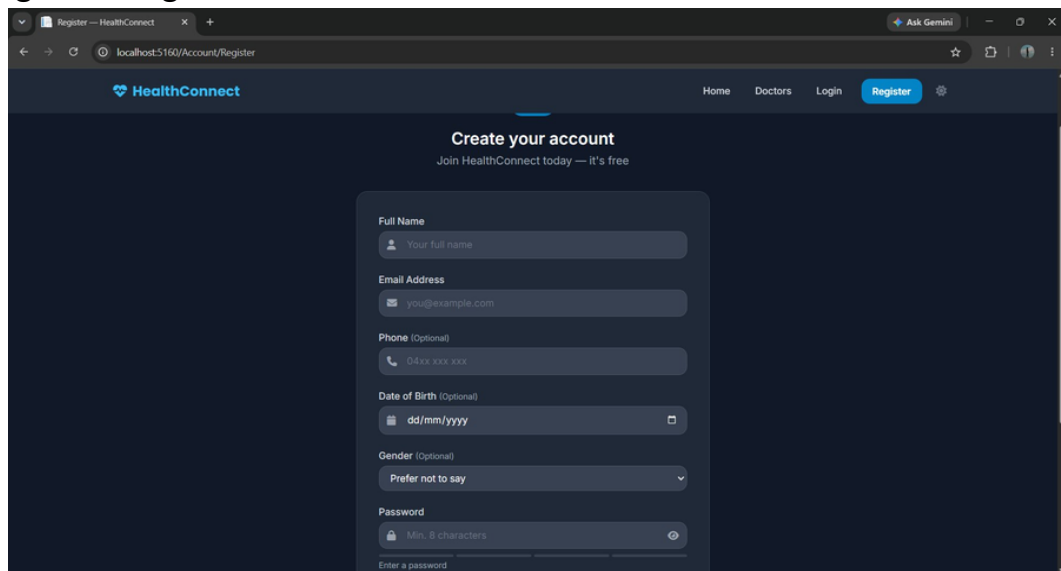


Figure 2. Registration form with real-time password strength indicator.

The registration form collects full name, email, phone, date of birth, gender, and password. Server-side validation using Data Annotations ensures all fields are correct before saving (Microsoft, 2024a).

2.3 Login Page

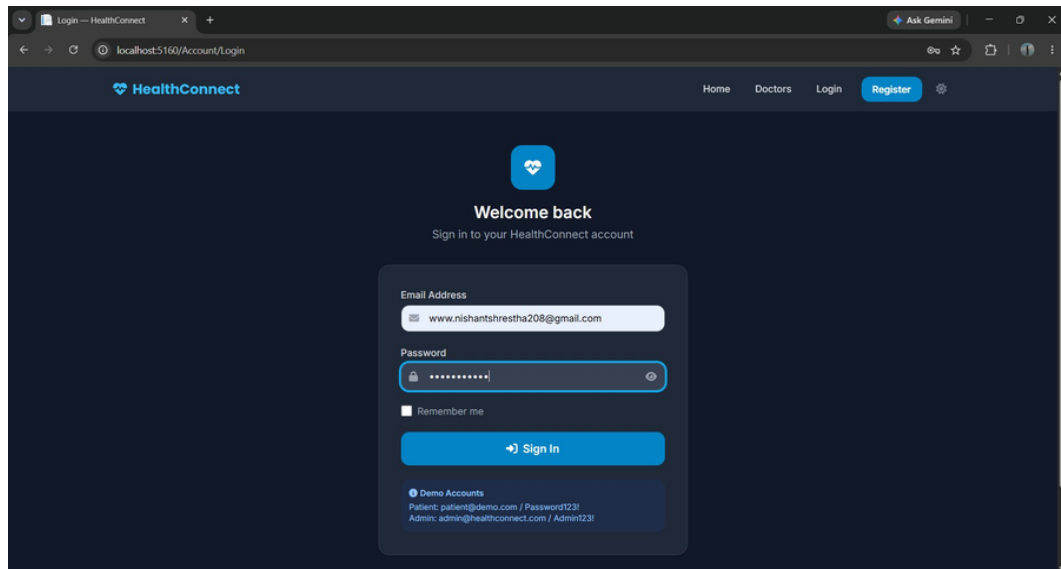


Figure 3. Login page with email, password fields, and show/hide toggle.

The login page uses ASP.NET Cookie Authentication as described by Microsoft (2024b). Passwords are verified against BCrypt hashes stored in the database (BCrypt.Net, 2023).

2.4 Patient Dashboard

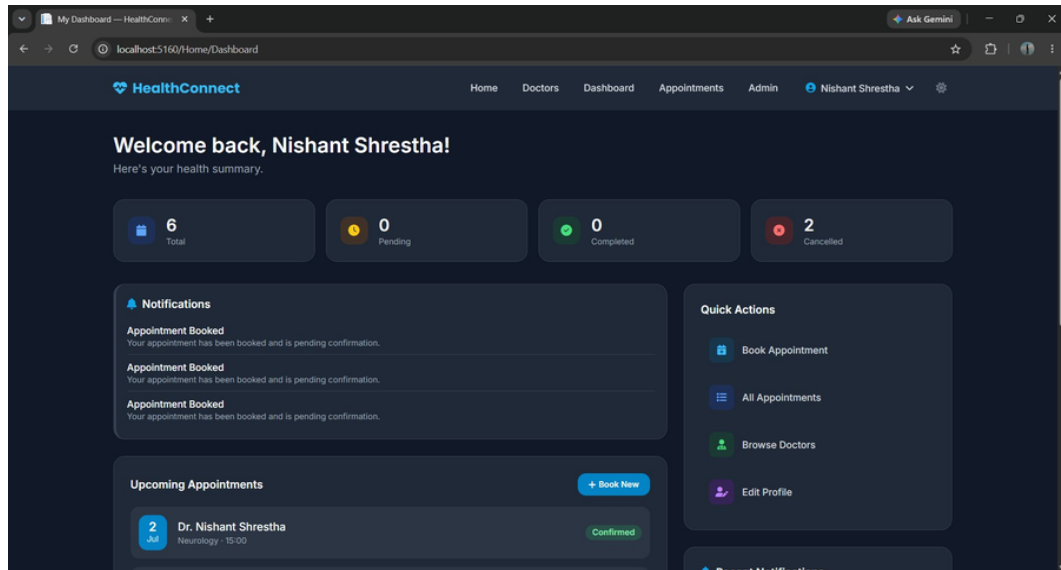


Figure 4. Patient dashboard showing appointment statistics and upcoming bookings.

The patient dashboard shows stat cards for total, pending, completed, and cancelled appointments along with a notifications panel and quick action links.

2.5 Admin Dashboard

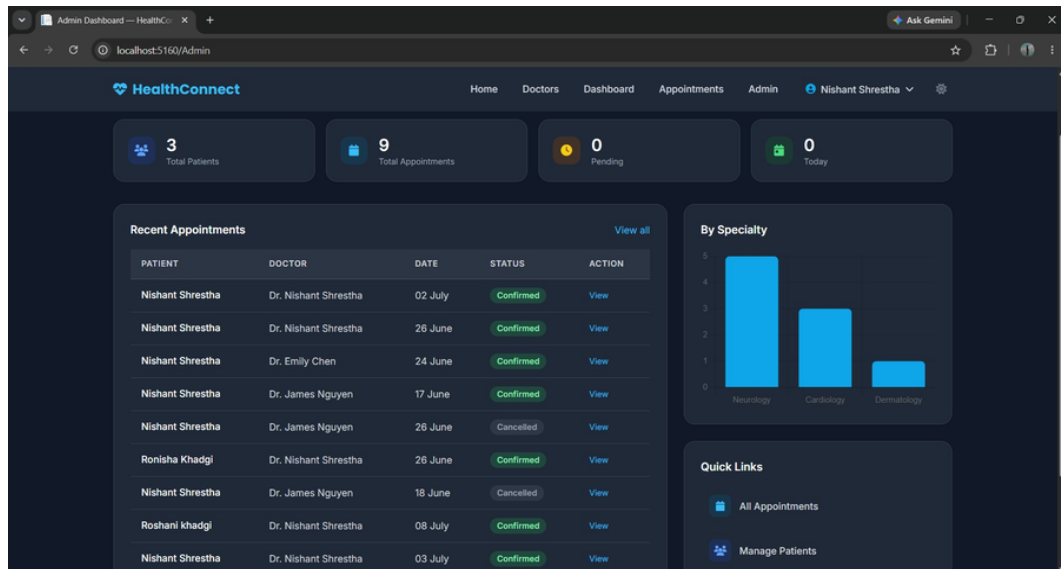


Figure 5. Admin dashboard with system statistics and Chart.js specialty bar chart.

The admin dashboard shows system-wide statistics and a bar chart showing appointments by specialty. The chart was built using Chart.js for interactive data visualisation.

2.6 Appointments List

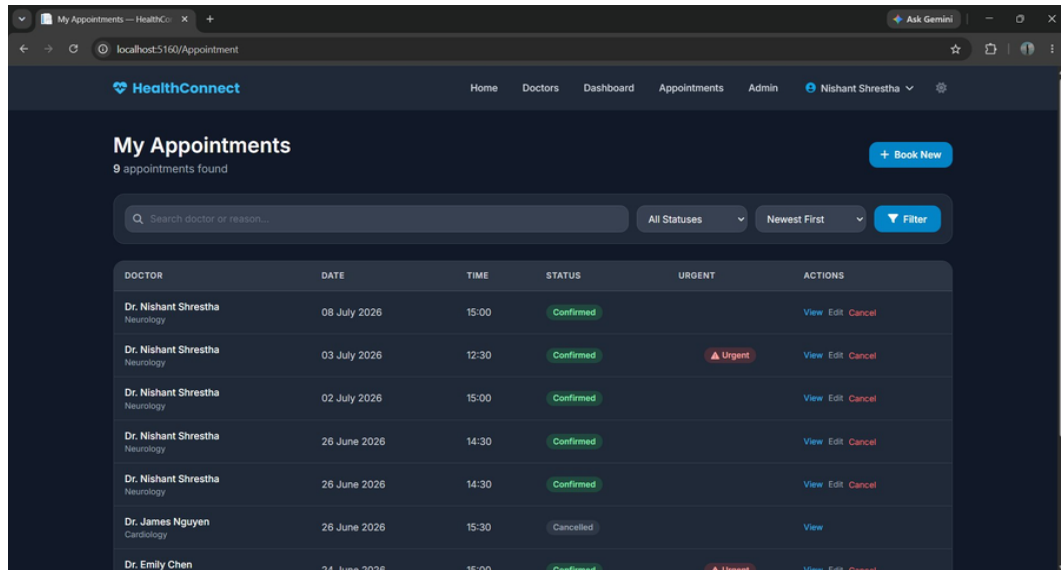


Figure 6. Appointments list with search, filter, status badges, and action buttons.

The appointments list shows all bookings in a responsive table with search and filter options. Frain (2020) recommends dual layout techniques for complex data on mobile, which is reflected in the card-based mobile view.

2.7 Doctors Page

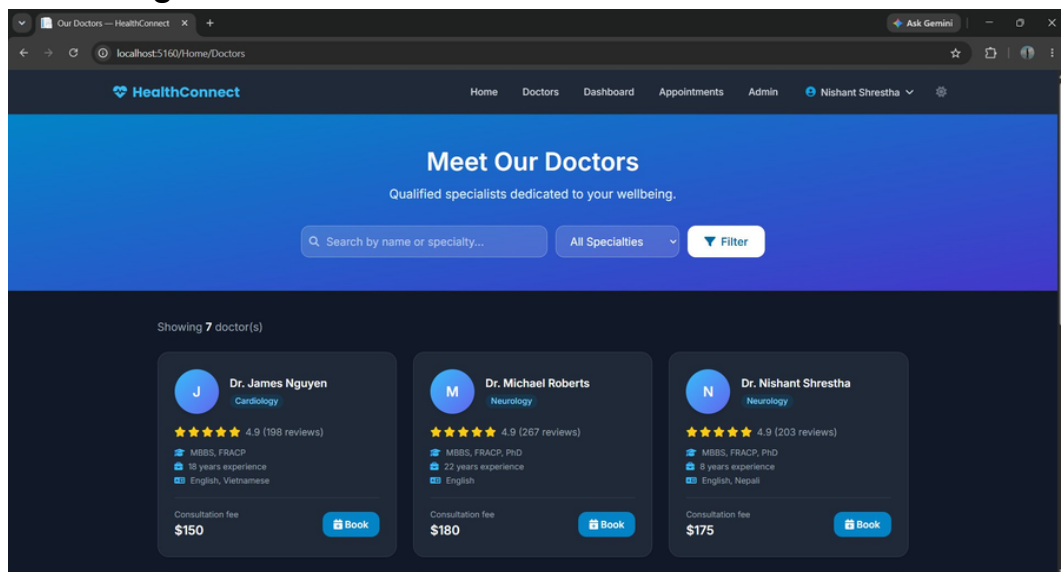


Figure 7. Doctors page showing specialist cards including Dr. Nishant Shrestha (Neurologist).

The doctors page shows specialists as cards with name, specialty, rating, experience, and fee. Dr. Nishant Shrestha is listed as a Neurologist with 8 years of experience and a 4.9 star rating.

3. Responsive Design

I made the website mobile-first, which means I designed for small screens first and then added styles for bigger screens. As Frain (2020) explains, a mobile-first approach ensures that content is prioritised for the smallest viewport before being progressively enhanced for larger screens. I used Tailwind CSS to handle this with responsive classes like `sm:`, `md:`, and `lg:` (Tailwind Labs, 2024).

3.1 How the Layout Changes

- Small screens like phones — everything stacks in one column. Navigation becomes a hamburger menu. Cards replace tables.
- Medium screens like tablets — two-column grids appear. Tables show in the appointments list.
- Large screens like desktops — three-column doctor grid, full sidebar on the dashboard, and wider tables.

3.2 Dual Layout for Appointments

On desktop the appointments page shows a full data table. On mobile the same data shows as cards so it is easy to read without scrolling sideways. I used Tailwind's `hidden md:block` and `md:hidden` classes to switch between the two layouts based on screen size (Tailwind Labs, 2024).

3.3 Hamburger Menu

The navigation collapses into a hamburger menu on small screens. I used JavaScript to handle the open and close toggle and added `aria-expanded` for accessibility so screen readers can announce whether the menu is open or closed (W3C, 2018).

3.4 Dark Mode

A dark mode toggle button is in the navigation bar. The preference is saved in `localStorage` so it persists across sessions. I had to rewrite the CSS from Tailwind's `@apply` format to plain CSS because the CDN version of Tailwind does not support `@apply` in external stylesheets (Tailwind Labs, 2024).

4. How I Built the Website

4.1 Project Structure (MVC)

I followed the Model-View-Controller (MVC) architectural pattern as implemented in ASP.NET Core 10. Microsoft (2024a) describes MVC as a design pattern that separates an application into three main components to improve testability and separation of concerns:

- Models — C# classes representing data such as Appointment with fields AppointmentDate, Status, DoctorName, and IsUrgent. ViewModels pass validated data between controllers and views.
- Views — .cshtml files using Razor syntax and Tag Helpers like asp-for and asp-controller to generate strongly-typed HTML.
- Controllers — Four controllers: AccountController for login and register, AppointmentController for booking CRUD, HomeController for the home page and dashboard, and AdminController for the admin panel.

4.2 The Database

I used SQL Server 2025 Express (Microsoft, 2025) with instance name NISHANT\SQLEXPRESS using Windows Authentication. The database was created manually in SSMS with the following tables: Users, Doctors, Specialties, Appointments, Notifications, and AuditLog. Dr. Nishant Shrestha was added as a Neurologist directly via an SQL INSERT statement.

4.3 Security

The OWASP Foundation (2021) identifies SQL Injection and broken authentication as two of the top web security risks. The following controls were implemented to address these and other threats:

- SQL Injection — SqlParameter is used for every database query so user input is never placed directly into SQL strings (OWASP Foundation, 2021).
- CSRF Protection — Every form includes a hidden anti-forgery token validated on every POST action (Microsoft, 2024a).
- Password Security — BCrypt hashing with work factor 11 ensures passwords are never stored in plain text (BCrypt.Net, 2023).
- Login Security — ASP.NET Cookie Authentication with an 8-hour sliding session. The cookie is HttpOnly to prevent JavaScript access (Microsoft, 2024b).
- Role Protection — The [Authorize(Roles='Admin')] attribute blocks unauthorised access. Patients attempting to access other patients' data receive a 403 Forbidden response.

5. Main Features

5.1 Registration and Login

Registration collects name, email, password, phone, date of birth, and gender. Server-side validation using Data Annotations checks every field before the record is inserted (Microsoft, 2024a). A uniqueness check prevents duplicate email addresses. Passwords are hashed using BCrypt with a work factor of 11, which Provos and Mazieres (1999) describe as a future-adaptable scheme that can be tuned to remain resistant to brute-force attacks.

Login verifies the submitted password against the stored BCrypt hash. On success, a Claims-based cookie is issued containing the user's ID, name, email, and role (Microsoft, 2024b). Suspended accounts are blocked with a clear error message.

5.2 Booking an Appointment

Patients select a doctor from a dropdown grouped by specialty. A JavaScript info card dynamically displays the doctor's fee without a page refresh. The server-side AppointmentValidator checks that the date is in the future, falls on a weekday, and is within 90 days. A duplicate booking query also prevents patients from booking the same doctor at the same time slot twice.

5.3 Admin Panel

The admin dashboard is protected with `[Authorize(Roles='Admin')]` (Microsoft, 2024a). It displays total patients, total appointments, pending count, and today's bookings. A Chart.js bar chart visualises appointment volume by specialty. Admins can search, filter, confirm, and cancel appointments, as well as suspend or reactivate patient accounts.

5.4 Doctors Page

The Doctors page shows all available specialists as cards with name, specialty, star rating, experience, languages, and consultation fee. Keyword search and specialty filtering use a dynamic SQL WHERE clause. Dr. Nishant Shrestha is listed as a Neurologist with 8 years of experience, a 4.9 star rating, and a \$175 consultation fee.

6. Accessibility

The Web Content Accessibility Guidelines (WCAG) 2.1 (W3C, 2018) were used as the reference standard for accessibility. Healthcare websites serve users across a wide range of abilities and digital literacy levels, making accessibility a critical design requirement (Australian Digital Health Agency, 2023). The following criteria were addressed:

- Skip to content link (WCAG 2.4.1) — keyboard users can bypass navigation and jump directly to the main content.
- ARIA labels (WCAG 4.1.2) — aria-expanded, aria-haspopup, and aria-live are used on interactive elements so screen readers announce state changes correctly (W3C, 2018).
- Colour contrast (WCAG 1.4.3) — all text meets the minimum 4.5:1 contrast ratio in both light and dark mode.
- Touch targets (WCAG 2.5.5) — all buttons and links are at least 44x44 pixels for mobile users.
- Error messages (WCAG 3.3.1) — validation errors appear directly below the relevant field in plain English.
- Status badges (WCAG 1.4.1) — badges use both colour and text labels so colourblind users can understand appointment status.
- Focus visible (WCAG 2.4.7) — all interactive elements display a visible outline when focused with a keyboard.

Older patients benefit from large text and generously sized buttons. People with low digital literacy benefit from the single-page booking form and plain error messages. Screen reader users benefit from semantic HTML5 elements including nav, main, section, and article (W3C, 2018).

7. Challenges Encountered

Several technical challenges arose during development and were resolved as follows:

- CSS compatibility — Tailwind's @apply directive is not supported in external stylesheets when using the CDN play build (Tailwind Labs, 2024). All component styles were rewritten in plain CSS to resolve this.
- Database connection — The application initially failed to connect because the connection string referenced a LocalDB instance that was not installed. This was resolved by installing SQL Server 2025 Express (Microsoft, 2025) and updating the connection string to point to NISHANT\SQLEXPRESS with Windows Authentication.
- Duplicate model class — The default ASP.NET Core MVC template generates an ErrorViewModel.cs file automatically (Microsoft, 2024a). A naming conflict arose because the same class was defined in Models.cs. The default file was deleted to resolve the build error.
- Dark mode input visibility — Input fields were invisible in dark mode because both the background and text colour were dark. Explicit CSS rules targeting dark mode were added to fix the contrast.
- Port conflict on restart — Running dotnet run while the application was already running caused an address already in use error. The taskkill command was used to terminate the existing process before restarting.
- BCrypt hash mismatch — The database seed script used placeholder BCrypt hashes that did not correspond to any real password (BCrypt.Net, 2023). A new account was registered through the application and the Role field was updated to Admin directly in SSMS.

8. References

Australian Digital Health Agency. (2023). *National digital health strategy 2023–2028*. Australian Government. <https://www.digitalhealth.gov.au>

BCrypt.Net. (2023). *BCrypt.Net-Next* (Version 4.0.3) [NuGet package].
<https://www.nuget.org/packages/BCrypt.Net-Next>

Frain, B. (2020). *Responsive web design with HTML5 and CSS* (3rd ed.). Packt Publishing.

Microsoft. (2024a). *Overview of ASP.NET Core MVC*. Microsoft Learn.
<https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>

Microsoft. (2024b). *Use cookie authentication without ASP.NET Core Identity*. Microsoft Learn.
<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie>

Microsoft. (2025). *SQL Server 2025 Express edition*. Microsoft Corporation.
<https://www.microsoft.com/en-au/sql-server/sql-server-downloads>

OWASP Foundation. (2021). *OWASP Top Ten*. Open Web Application Security Project.
<https://owasp.org/www-project-top-ten/>

Provos, N., & Mazieres, D. (1999). A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*, 32, 81–91. USENIX Association.

Tailwind Labs. (2024). *Tailwind CSS documentation* (Version 3). <https://tailwindcss.com/docs>

W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium.
<https://www.w3.org/TR/WCAG21/>